



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Processamento de Linguagens

Ano Letivo de 2025/2026

Construção de um Compilador para Fortran 77 Standard

Carlos Araújo
A106845

Vitor Senra
A106921

April 28, 2026

PL

Índice

1. Introdução	1
2. Arquitetura do Compilador	1
3. Análise Léxica	1
4. Análise Sintática	1
5. Análise Semântica e Tabela de Símbolos	1
6. Representação Intermédia (AST)	2
7. Geração de Código	2
8. Testes e Validação	2
9. Conclusões	2

1. Introdução

A disciplina de Processamento de Linguagens foca no estudo de compiladores. Este projeto implementa um compilador para **Fortran 77** que gera código para a VM da disciplina, usando Python 3 e a biblioteca PLY. O compilador segue uma pipeline modular: pré-processamento, análise léxica, sintática, semântica e geração de código. O relatório cobre arquitetura, implementação e testes.

2. Arquitetura do Compilador

O compilador tem módulos independentes em `src/`: `lexer.py` (tokenização), `parser.py` (análise sintática + AST), `ast_nodes.py` (nós da árvore), `symbol_table.py` (tabela de símbolos) e `codegen.py` (geração VM). O `main.py` orquestra o pipeline: lê `.for`, normaliza formato fixed-format, gera tokens com `ply.lex`, constrói AST com `ply.yacc`, e emite instruções VM. A separação de responsabilidades facilita manutenção e teste.

3. Análise Léxica

O `lexer.py` usa `ply.lex` e trata o formato fixed-format Fortran 77: colunas 1-5 (labels), 6 (continuação/comentário), 7-72 (código). O pré-processador `FortranPreprocessor` une linhas e remove comentários antes da tokenização.

Tokens: palavras-chave (`PROGRAM`, `IF`, `DO`, etc.), operadores (`.GT.`, `**`), literais (inteiros, reais, strings, `.TRUE.`/`.FALSE.`) e identificadores (até 6 caracteres, maiúsculas). O lexer é case-insensitive. Erros léxicos lançam `LexerError` com linha de origem.

4. Análise Sintática

O `parser.py` usa `ply.yacc` (LALR(1)) com gramática BNF em `CFG.md`. A precedência de operadores é garantida pela hierarquia: `logical_or` $\rightarrow$ `logical_and` $\rightarrow$ `relational` $\rightarrow$ `additive` $\rightarrow$ `multiplicative` $\rightarrow$ `power` $\rightarrow$ `unary` $\rightarrow$ `primary`.

O parser integra verificações semânticas: duplicados, variáveis não declaradas, dimensões de arrays, labels em ciclos `DO`, e saltos `GOTO`. Cada regra constrói nós da AST (`ast_nodes.py`). Erros sintáticos lançam `SyntaxError`, semânticos lançam `SemanticError`.

5. Análise Semântica e Tabela de Símbolos

O `symbol_table.py` implementa escopos aninhados (pilha de dicionários). Cada `Symbol` tem: nome, tipo (`variable/array/function`), `dtype` (`INTEGER/REAL/LOGICAL/CHARACTER`), dimensões e linha.

Verificações: declarações duplicadas (`declare()`), variáveis não declaradas (`lookup()`), acessos inválidos a arrays (`check_array_access()`), e compatibilidade de tipos (`is_type_compatible()`). O Fortran 77 permite conversões implícitas (ex: REAL para INTEGER trunca).

6. Representação Intermédia (AST)

A AST em `ast_nodes.py` usa `dataclass`. Nós principais: `Program`, `CompilationUnit`, `TypeDeclaration`, `AssignmentStatement`, `IfThenElse`, `DoLoop`, `GotoStatement`, `BinaryOp`, `UnaryOp`, `Variable`, `ArrayAccess`, `FunctionCall`, `Literal`.

A AST separa análise sintática da geração de código. O `codegen.py` percorre a árvore para gerar instruções VM, podendo aplicar otimizações antes (dobragem de constantes, código morto).

7. Geração de Código

8. Testes e Validação

Cinco programas de teste:

- `hello.for`: PRINT de strings
- `fatorial.for`: DO, IF, GOTO, inteiros
- `primo.for`: Ciclos aninhados, lógica
- `somaarr.for`: Arrays (DIMENSION)
- `conversor.for`: Reais, READ

9. Conclusões